

Understanding Neural Networks from the Ground Up: A NumPy-Only Implementation Compared with a Framework Baseline

Abstract—Deep learning frameworks such as TensorFlow and Keras make neural networks easy to build but hide the mechanics of forward propagation, backpropagation, and optimization. This project implements a fully connected feedforward neural network using only NumPy, including layers, activation functions, gradient computation, weight initialization, and optimizers (SGD, momentum, and Adam). We verify our gradients using numerical gradient checking. We then build an identical architecture in [TensorFlow/Keras or PyTorch] and compare both implementations on the Fashion-MNIST dataset in terms of test accuracy, convergence behavior, training time, and sensitivity to hyperparameters. Our goal is to learn how these systems work internally and to explain why the two implementations behave similarly or differently.

I. INTRODUCTION

Modern frameworks let a practitioner train a network in a few lines of code. This convenience hides how gradients are computed, how initialization affects training, and what the optimizer is doing. Students who only use high-level APIs often cannot explain why a model fails to converge or why two seemingly identical models give different results.

This project addresses that gap. Our contributions are:

- 1) A NumPy-only feedforward neural network with modular layers and multiple optimizers.
- 2) Verification of our backpropagation using numerical gradient checking.
- 3) A controlled comparison against a framework implementation of the same architecture.
- 4) An analysis of the sources of any differences, such as default initializers, numerical precision, and vectorization.

II. PROBLEM STATEMENT

Research question: *How does a neural network implemented from scratch using only basic numerical libraries compare with an equivalent framework-based model in accuracy, training behavior, and computational efficiency, and what factors explain the differences?*

Objectives:

- Implement forward and backward passes from first principles.
- Validate the implementation through gradient checking.
- Compare against a framework model under controlled conditions.

III. RELATED WORK

Reference suggestions were obtained with an AI assistant [6]. *Backpropagation.* Rumelhart et al. [1] showed that gradient-based learning through backpropagation can train multilayer networks to learn useful internal representations.

Initialization. Glorot and Bengio [2] analyzed how weight scale affects signal flow in deep networks, and He et al. [3] extended the analysis to ReLU activations.

Optimization. Kingma and Ba [4] introduced Adam, which combines momentum with adaptive learning rates.

Data and benchmarks. Street et al. [5] described the nuclear-feature extraction behind the Breast Cancer Wisconsin (Diagnostic) dataset used in our experiments. [Add 2–3 sources on feature scaling for neural networks and on from-scratch tutorials or framework benchmarking.]

Gap. Existing work typically presents either theory or framework usage. We provide a side-by-side, controlled comparison that explains *why* the two implementations behave as they do.

IV. PROPOSED METHODOLOGY

A. Dataset

We use the Breast Cancer Wisconsin (Diagnostic) dataset: 569 samples, each described by 30 numeric features computed from a digitized image of a fine-needle aspirate of a breast mass, labeled benign (357) or malignant (212). We use a stratified 70/15/15 split into training, validation, and test sets so that the class ratio is preserved. Each feature is standardized using statistics from the training set only, $x' = (x - \mu_{\text{train}})/\sigma_{\text{train}}$, to avoid data leakage. Because the classes are imbalanced and missing a malignant case is the costly error, we report precision, recall, and F1 for the malignant class in addition to accuracy. With only 569 samples, a single split is noisy, so we repeat all experiments over multiple random seeds. We also test the effect of feature scaling by training with and without standardization.

B. Architecture

The network is Input ([2,000] TF-IDF features) $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dense(2, Softmax), shown in Fig. 1.

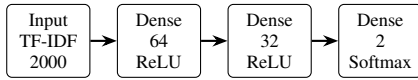


Fig. 1. Feedforward architecture used in both the NumPy and framework implementations.

C. Forward Pass

For layer l with weights $W^{(l)}$ and biases $b^{(l)}$:

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad (1)$$

$$a^{(l)} = f(z^{(l)}), \quad (2)$$

$$\hat{y}_k = \frac{\exp(z_k)}{\sum_j \exp(z_j)}. \quad (3)$$

D. Loss and Backpropagation

We use the cross-entropy loss

$$L = - \sum_k y_k \log \hat{y}_k. \quad (4)$$

For a softmax output with cross-entropy, the output error simplifies to $\delta^{(L)} = \hat{y} - y$. Errors propagate backward as

$$\delta^{(l)} = \left(W^{(l+1)}\right)^\top \delta^{(l+1)} \odot f'(z^{(l)}), \quad (5)$$

and the gradients are

$$\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} \left(a^{(l-1)}\right)^\top, \quad \frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}. \quad (6)$$

E. Initialization and Optimizers

We implement Xavier and He initialization, and SGD, SGD with momentum, and Adam. Updates use mini-batches of size [64].

F. Gradient Checking

We compare analytic gradients to the centered finite difference

$$\frac{\partial L}{\partial \theta} \approx \frac{L(\theta + \epsilon) - L(\theta - \epsilon)}{2\epsilon}, \quad (7)$$

and require a relative error below 10^{-7} .

G. Framework Baseline

We build the same architecture in [Keras/PyTorch] with matched initializers, optimizer settings, batch size, epochs, and random seeds.

V. EVALUATION PLAN

Table I lists the metrics we will report. All experiments use identical data splits and hyperparameters on both sides.

VI. EXPECTED OUTCOMES

Through this experiment, we expect to have a deeper understanding of neural networks so that not only can we make our own neural networks using libraries but also trace any problems we face during working and understand how each part of the neural network works from the ground up. Our ultimate goal is to make a neural network with transformers that can rival those made using pre-existing frameworks.

TABLE I
PLANNED EVALUATION METRICS

Aspect	Metric
Accuracy	Test accuracy, malignant-class precision/recall/F1, confusion matrix
Convergence	Training/validation loss and accuracy curves
Efficiency	Seconds per epoch, total training time

REFERENCES

- [1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.
- [2] X. Glorot and Y. Bengio, "Understanding the difficulty of training deep feedforward neural networks," in *Proc. AISTATS*, 2010.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. ICCV*, 2015.
- [4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.
- [5] W. N. Street, W. H. Wolberg, and O. L. Mangasarian, "Nuclear feature extraction for breast tumor diagnosis," in *Proc. SPIE*, vol. 1905, 1993, pp. 861–870.
- [6] Anthropic, "Claude Sonnet 5.5," large language model, San Francisco, CA, USA, 2026. [Online]. Available: <https://claude.ai>. Accessed: Oct. 4, 2026.